real estate by mckenna Development Handbook

Architecture usage troubleshooting and continuing development

OchsSoft | 26 October 2026

This handbook explains the project from everyday use to the code and data that implement it. Use it to set up a development checkout, follow a request or document through the system, locate the right extension point, make a change, and verify the result.

Begin with the project overview and architecture. The later chapters provide exact local commands, user journeys, worked development examples, diagnostics, recovery steps, known gaps, and a pinned source reference. Technical terms are explained where they are used.

The documented source is OchsSoft/real-estate-by-mckenna at 3f38e3528bd1c679f50ae23f34a0b87050623e5b. Source-defined behavior, observed evidence, planned capabilities, and remaining platform work are distinguished. The quick operations guide remains a separate short reference.

Contents

Understand the project

Understand the project 3
Architecture 4

Use and develop the project

Local development 7
Using and running 8
Developing and testing 9

Diagnose improve and verify

Gaps and recovery 10
Source reference 11

Understand the project

The product and its current limits

This repository contains a public real-estate portfolio and contact page. Visitors can learn about McKenna, view selected property-marketing examples, read the site's positioning, and follow contact, scheduling, or social links. It is a static website: its authored HTML, styles, images, and browser script are delivered directly to the visitor. There is no application server that processes leads and no administrative dashboard hidden behind the page.

The entire interactive implementation is in `index.html`. Assets under `assets/` provide portrait, listing, and supporting imagery. A small slideshow rotates through eight authored slide entries. Its data is editorial content in JavaScript, not a live listing feed. A sold or under-contract headline reflects the checked-in text; it does not query a real-estate service or verify current availability.

This is an independent business website. It does not require the AIOS stack to start or remain useful. If operational visibility is added later, keep the website's public content and client-facing identity independent from private internal tools.

What visitors can expect

The page has navigation anchors for top, about, work, and contact. The work section presents a sold-home slideshow with previous, next, and dot controls. Contact actions hand off to email, an external scheduling service, or a social site. The website does not itself record that an inquiry was sent or an appointment was booked.

Capability	State at this baseline
Portfolio and brand content	Authored HTML and images.
Carousel	Browser JavaScript with timed and manual navigation.
Lead form/database	Absent.
Live MLS or listing synchronization	Absent.
Scheduling	External link; destination owns completion.
User accounts or payments	Absent.
Build system/package installation	Absent.

A static site is not inherently unfinished. It is a good fit for a small brochure or portfolio when contact is intentionally external. Its limitations need to be clear so future development adds only the capabilities the business actually needs.

Reading with implementation confidence

This handbook explains the committed implementation and safe continuation work. It does not attest that every external destination is currently available or that the live domain serves this exact revision. Verify those separately during a release review. The source reference pins this edition so you can tell which facts came from code and which are proposed improvements.

Architecture

Page assets and external services

`index.html` contains three concerns: document content, CSS in a style block, and JavaScript in a script block near the end of the body. This keeps deployment simple but makes the file a shared editing surface. A layout change can be close to a carousel change, so review the diff carefully and avoid broad formatting rewrites during a small content correction.

The static host only needs to serve files with appropriate content types. The browser renders HTML and CSS, fetches referenced images, and executes the slideshow. Contact and scheduling links navigate to other applications. There are no API routes, webhooks, environment variables, database schemas, or background jobs in this repository.

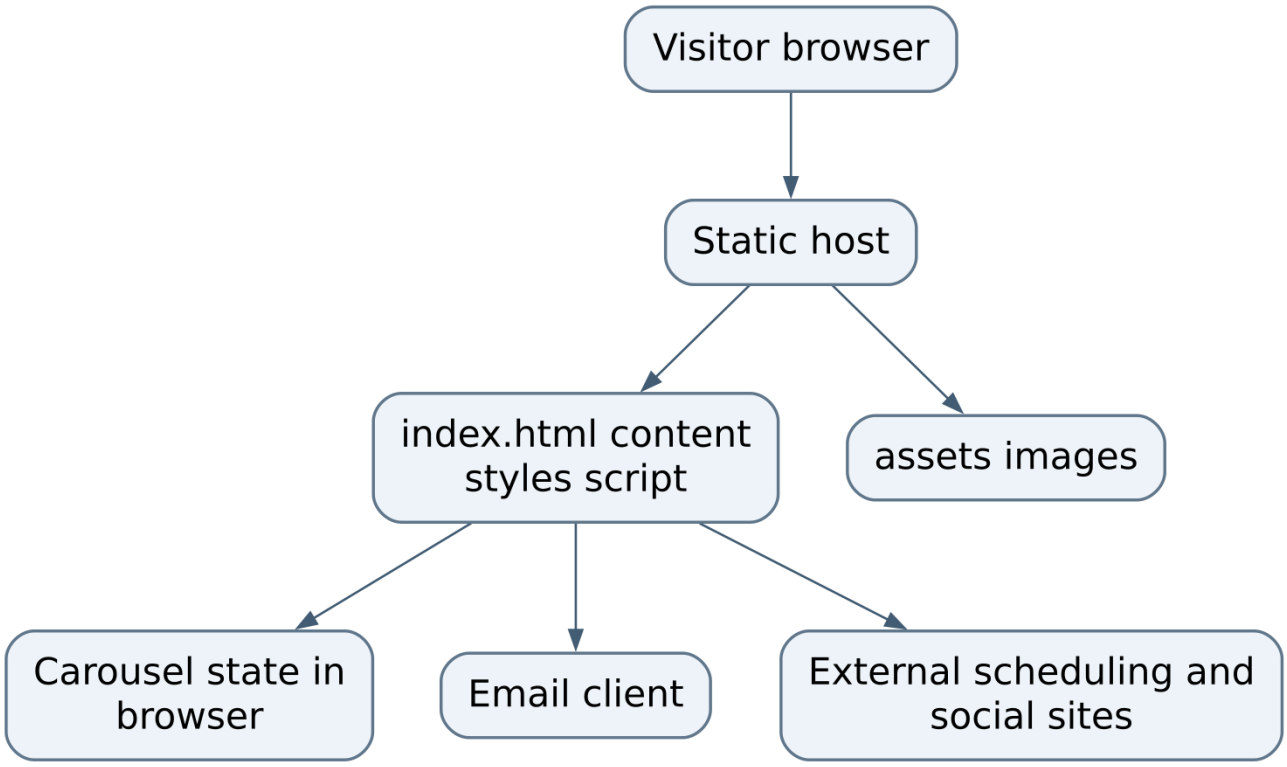


Figure 1 Page, assets, and external services

Carousel data contract

The `slides` array contains objects with `image`, `alt`, `title`, `text`, and `url`. The image path is relative to the page. `alt` provides an accessible description. `title` and `text` describe the selected example. `url`

links to its external source. Keep these fields describing the same property or example; an image and headline mismatch is a content defect even when the code runs.

`activeSlide` stores the current array index. `setSlide(index)` wraps the index with modulo arithmetic so previous on the first slide moves to the last. It then writes the image source and alternative text, visible title and text, and destination link, before rebuilding the dot controls.

`renderDots()` clears the dot container and creates one button per slide. Each has a `Show photo N` accessible label and `aria-current` reflecting the selected state. Clicking a dot calls `setSlide` and restarts the timer.

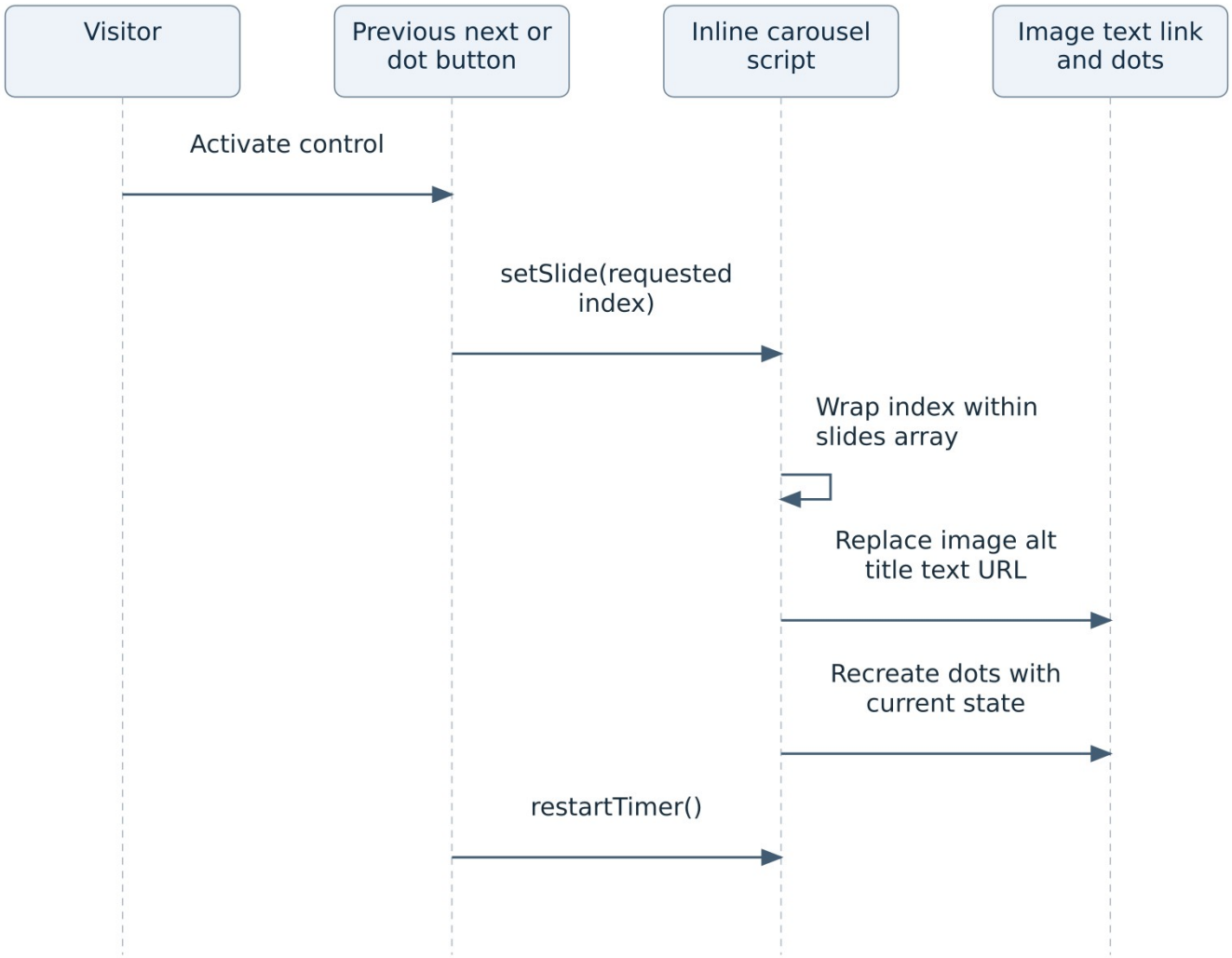


Figure 2 Carousel data contract

Timer and accessibility boundary

`restartTimer()` clears the previous interval. Unless `prefers-reduced-motion: reduce` matches, it schedules the next slide every 6500 milliseconds. Manual navigation also restarts that interval. Startup calls `setSlide(0)` and then `restartTimer()`.

The reduced-motion check happens when restarting; a preference change while the timer is already running is not subscribed to. The script also recreates dot buttons each time, so keyboard focus behavior deserves manual attention when using a dot. There is no explicit pause control or visibility-change handler. These are concrete usability improvement candidates, not reasons to replace the entire site framework.

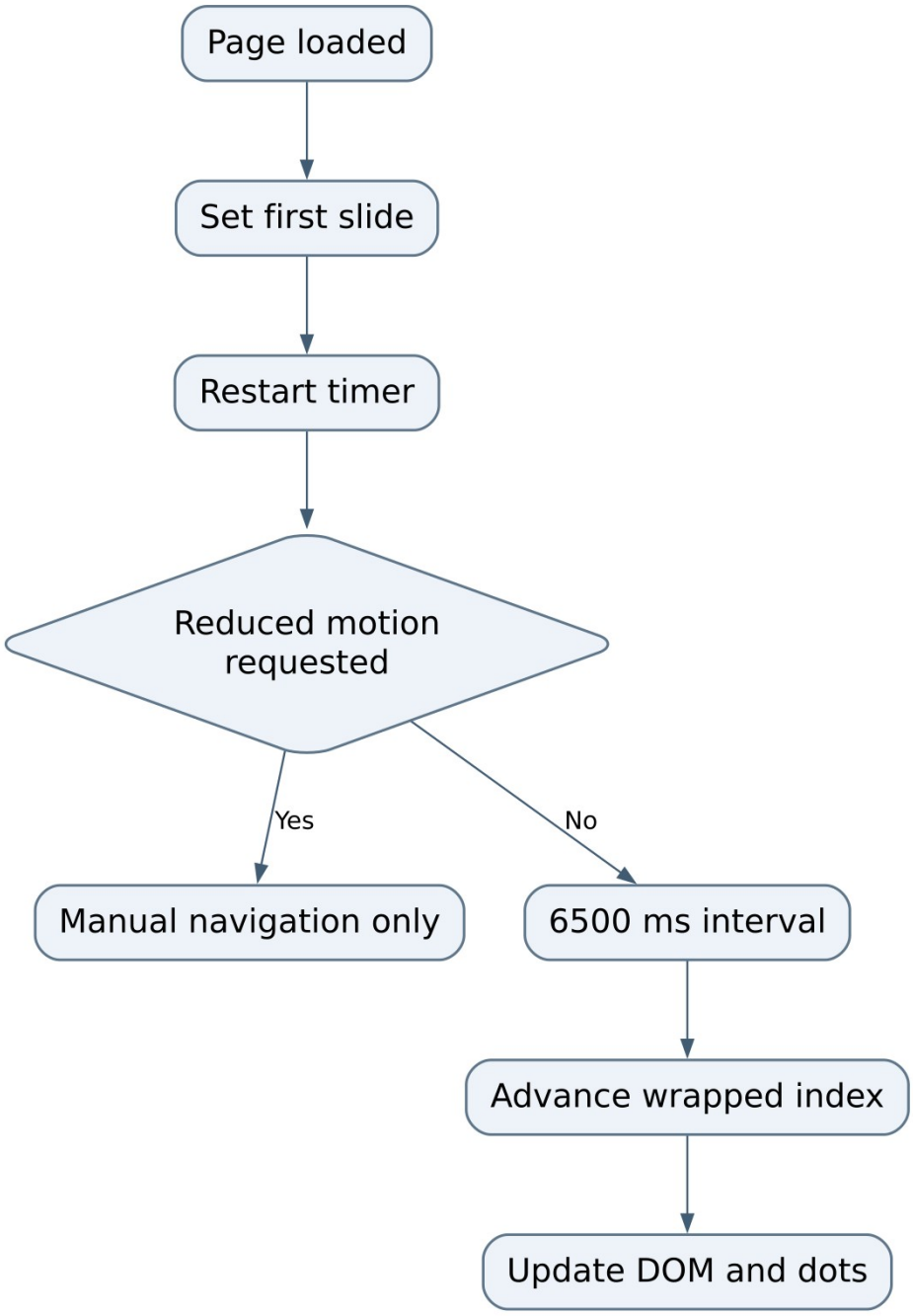


Figure 3 part 1 of 2 Timer and accessibility boundary

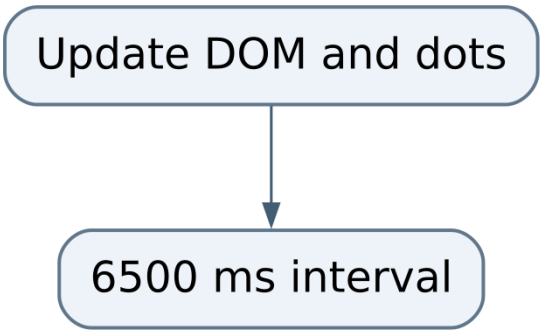


Figure 3 part 2 of 2 Timer and accessibility boundary

Ownership of information

Git owns the approved website source and static media. The visitor's browser owns temporary slideshow position. External providers own appointment and social interactions. An email client owns composing and sending a message. There is no internal lead record to back up or query. A future CRM integration must introduce an explicit consent, delivery, retention, and error contract rather than treating an outbound link click as a completed sale inquiry.

Local development

Prepare and preview

Inspect the repository's branch, status, and current commit before editing. Work on a topic branch from updated `master`, keeping unrelated local edits intact. This baseline has no package manifest, dependency lockfile, build command, or framework CLI. Do not run `npm install` as a default step; it would add machinery the site does not use.

Use a native static server from the repository root:

```
python3 -m http.server 4173 --bind 127.0.0.1
```

Open `http://127.0.0.1:4173/`. This is a maintenance preview command for the static files, not a repository-provided application daemon. If Python is unavailable on NixOS, enter a temporary Nix environment containing Python 3 rather than installing a global runtime or adding a container. The repository has no pinned Nix flake, so full reproducibility and production-host qualification remain gaps.

The server must run from the directory containing `index.html`; otherwise relative assets may return 404 or you may see a directory listing. Check the terminal's requests and browser Network panel. The application needs no secret values or environment variables. Port 4173 is a preview choice and can be changed if occupied.

Local evidence to collect

Because there is no automated suite, a content or interface change needs a deliberate local inspection. Load the page at desktop and narrow phone widths. Check that the header, about section, work examples, and contact actions are reachable. Use keyboard navigation to reach slideshow controls. Inspect the console for exceptions and Network panel for missing images. Advance through all slides and wrap in both directions.

Set reduced motion through the browser or operating system and reload; automatic rotation should stop while controls still work. Do not actually submit an appointment or send a message as a casual test. Validate the destination URL and use an approved non-production verification path if completion must be tested.

These observations are not generated by a build. Record the actual browser and tested viewports in the PR. If adding automated checks later, use a small tool appropriate to static HTML and browser behavior; avoid introducing a large application runtime merely to validate links.

Publishing boundary

The repository does not define a complete deployment pipeline. A local HTTP preview does not prove HTTPS, cache headers, redirects, domain records, or the live host's behavior. Keep deployment credentials outside the repo. Push a reviewed topic branch and open a PR against `master`; human merge and release remain separate actions. Preserve deployment workflows if added later, while keeping routine hosted CI disabled unless explicitly authorized.

Using and running

Visitor journeys

A visitor looking for background should use the About navigation link and read the agent introduction. A visitor looking for proof of work should use the Work section, inspect a slide's image and summary, and follow its source link if desired. A visitor ready to talk should use the contact area and choose email or the scheduling destination.

The carousel is editorial: it shows selected examples rather than an inventory search. Its next and previous controls loop continuously through the array. The dots allow direct selection. The visible caption and outgoing link should change with the image. If only the image changes or the source link points to a different example, stop treating the page as correct until the corresponding fields are reconciled.

An external scheduling page may ask for information or show availability. That experience belongs to the external service. The local site cannot promise an appointment exists until that service confirms it. Similarly, clicking email opens a composition action; the visitor must still send the message.

Maintainer run and close

Start the Python preview in a dedicated terminal and keep its URL visible. Reload after source changes; there is no hot-reload service. If styles appear unchanged, verify you edited the served checkout and inspect the response before clearing all browser state.

Close the tab and stop the preview with Ctrl+C in its owning terminal. There is no database flush, queue drain, worker shutdown, or file upload in progress. Do not stop other projects' servers by killing every Python process or every listener on a guessed port.

Backups and restore

The source repository and its static image assets are the complete local site artifact. Keep approved images in source control when their size is reasonable and usage rights are clear. Do not substitute

private client documents or unapproved imagery into a public asset directory. A preview server exposes files under its root, so keep credentials and unpublished customer records outside that tree.

To recover from a broken change, use Git history to identify the last reviewed content and restore the affected file through a new scoped change. Preserve newer unrelated work. Recheck all relative asset paths and slideshow entries after the restore. There is no runtime database to restore, and rolling back the site does not cancel appointments held by an external service.

Developing and testing

Recipe add or replace a work example

Prepare an approved web image under `assets/` with a descriptive stable filename. Optimize it before committing; an oversized photo affects every visitor. Add a `slides` entry containing all five fields. Write alternative text that describes the useful visual information rather than repeating “image of.” Keep status and narrative accurate for the example and confirm the source destination is intended for public visitors.

Load the preview, navigate directly to the new dot, and verify image, alternative text in the DOM, title, caption, and URL together. Test first-to-last and last-to-first wrapping after changing the array length. Wait for automatic rotation with normal motion settings and repeat with reduced motion. A missing array comma can prevent the entire script from parsing, so inspect the console.

Recipe change contact or positioning

Search `index.html` for all copies of the old text or destination. Visible contact wording, link URLs, headings, and metadata can diverge if only one occurrence changes. Update the intended fields together without altering unrelated property stories. Review external links for correct scheme and destination; `mailto:` is an email action, not an HTTPS page.

For a scheduling replacement, agree on the approved external endpoint first. The current implementation needs only a link change. Do not add a client-side API token or collect inquiries in an improvised browser store. If an embedded form becomes necessary, design success, failure, spam control, privacy, retention, and accessible errors as a separate feature.

Recipe separate the one file implementation

If the inline style or script becomes difficult to maintain, extract it into dedicated assets without changing behavior in the same step. Move CSS to a stylesheet and the slideshow to a script that loads after its required elements. Keep relative asset resolution equivalent; JavaScript strings used as DOM image URLs resolve relative to the page, which is easy to confuse with module-relative paths.

Acceptance should compare the page before and after extraction: layout, all eight slides, keyboard controls, reduced motion, and contact links. A refactor is complete when the observable behavior is preserved, not merely when new files exist. Avoid adding a framework until there is a concrete product need that outweighs its install and build complexity.

Recipe improve carousel access

Introduce a visible pause/resume control and preserve the user's choice across manual navigation within the page. Consider listening for changes to reduced-motion preferences and pausing while the

page is hidden. Maintain stable dot elements instead of recreating the focused element on every slide change.

Test with keyboard only: focus a dot, activate it, and confirm focus remains predictable. Pause, wait longer than 6500 milliseconds, and confirm the slide stays fixed. Resume, verify rotation, then enable reduced motion and confirm the stronger preference wins. These are proposed acceptance criteria; the existing baseline has no automated tests for them.

Review and maintainability

Keep source files reasonably below the workspace's 1000-line threshold as features grow. This baseline's single file is already substantial, so extraction is a practical split plan before adding a larger gallery or lead interface. Preserve the site's simple static deployment and explain any new dependency or service in this handbook. A small site should stay understandable without reconstructing an undocumented toolchain.

Gaps and recovery

Symptom diagnosis

Symptom	Evidence to inspect	Safe action
Page is a directory listing	Server root and requested URL.	Start preview from repository root.
Image is broken	Network 404, exact filename and case.	Repair the slide path or restore the approved asset.
Carousel never changes	Console parse error, missing DOM ID, reduced-motion preference.	Fix the specific script error; manual-only behavior under reduced motion is expected.
Caption and image disagree	The selected slide object and `setSlide`.	Correct the complete entry as one editorial change.
Focus disappears after dot activation	`renderDots` replaces the focused element.	Preserve stable controls in a scoped accessibility fix.
Email or scheduler did not complete	External application confirmation.	Explain the handoff and verify its destination; do not claim a saved local lead.
Live site differs from preview	Deployed revision and host cache.	Identify the released source before changing content again.

Prioritized gaps

Accessible slideshow control: `restartTimer` has a reduced-motion startup check but no explicit pause control and `renderDots` rebuilds controls. Add stable focus, pause/resume, and preference-change handling. Acceptance is a keyboard journey and a timed pause demonstration.

Repeatable checks: there is no automated test or pinned Nix environment. Add a small deterministic static/browser validation path, then demonstrate it on NixOS. Keep external scheduling and email submission outside routine tests.

Maintainability of one file: content, CSS, and behavior share `index.html`. Extract concerns before the file exceeds maintainable size; preserve all DOM contracts and compare the rendered page locally. Avoid mixing a mechanical split with a redesign.

Release documentation: no complete host/deployment contract is established in source. Document the actual host, artifact root, domain ownership, rollback process, cache behavior, and human release gate when verified. Do not infer these from a public domain alone.

Content maintenance: static success stories can become stale. Assign an editorial review cadence and require approval for changes in property status or claims. This is separate from a technical uptime check.

Things to avoid during repair

Do not add a database because a contact link did not send email. Do not rewrite the site because one image path is wrong. Do not expose private customer or internal stack data to make a public page appear more complete. Work from the failing layer, use the smallest complete repair, and retain real preview evidence with the review.

Source reference

This edition follows `OchsSoft/real-estate-by-mckenna` at committed revision `3f38e3528bd1c679f50ae23f34a0b87050623e5b`. Line links below point to that exact revision, so later changes cannot silently change the meaning of a reference. The handbook explains source behavior and identified gaps; it is not a claim that a new installation or production workflow was exercised.

Read the implementation

- index.html at line 11: Inline visual system and responsive rules.
- index.html at line 660: Portfolio section and carousel DOM contract.
- index.html at line 754: Authored slide record schema and public content.
- index.html at line 823: Accessible dot creation and focus-replacement boundary.
- index.html at line 839: Wrapped index and synchronized image/caption/link update.
- index.html at line 850: 6500 millisecond interval and reduced-motion guard.
- README.md at line 1: Minimal repository entry point and existing quick guide.

Verify the edition

Before following a development recipe, compare the current checkout with this revision. Read `git status --short` to identify existing edits and `git log -1 --oneline` to identify the current commit. A newer primary branch may intentionally change commands, schemas, or routes. Reconcile those

changes in the corresponding chapter before relying on an old instruction. Preserve the quick operations guide as the shorter entry point.

Source inspection does not establish end-to-end acceptance. Test names and commands describe evidence that a maintainer can obtain. Keep actual execution results beside the review for the change being made, and distinguish a skipped or blocked test from a passing one. Runtime data, credentials, build directories, and generated test output do not belong in source control.